

친구 - 풀이

작성자: 박선재

부분문제 1

$N \leq 3000$ 이므로, $1 \leq i, j \leq N$ 이고 $i \neq j$ 인 모든 순서쌍 (i, j) 에 대해 i 번 학생과 j 번 학생이 친구인지 알 수 있으므로, 모든 학생의 친구 수를 계산할 수 있다.

부분문제 2

학생들을 집의 좌표 순으로 정렬한다. 좌표가 모두 다른 정수이고 $K_1, K_2 \leq 10$ 이므로, 어떤 학생과 친구가 될 수 있는 학생은 정렬된 순서에서 양옆으로 각각 최대 10명뿐이다. 따라서 각 학생마다 양옆 10명씩만 확인하면 친구의 수를 정확하게 계산할 수 있다.

부분문제 3

좌표가 1 이상 N 이하의 정수이므로 좌표를 인덱스로 하는 배열을 쓸 수 있다. 학교별로 좌표에 학생 수를 기록한 누적합 배열을 만들면, 각 학생에 대해 자신과 같은 학교에 다니면서 거리가 K_1 이하인 학생의 수, 다른 학교에 다니면서 거리가 K_2 이하인 학생의 수를 빠르게 계산할 수 있다.

부분문제 4

모든 학생이 같은 학교 소속이므로, 각 학생의 친구 수는 자기 자신을 제외하고 집 사이의 거리가 K_1 이하인 학생의 수와 같다. 모든 학생의 집 위치를 정렬하고, 이분 탐색을 사용하면 각 학생의 친구 수를 빠르게 계산할 수 있다.

부분문제 5

모든 학생이 2개의 학교 중 하나에 다니므로, 각 학교에 대해 해당 학교에 다니는 학생들의 집 좌표들을 따로 모은 후, 각각 정렬한다. 이후 부분문제 4와 비슷하게 이분 탐색을 사용하면 각 학생의 친구 수를 빠르게 계산할 수 있다.

부분문제 6

자신과 다른 학교를 다니면서 집의 거리가 K_2 이하인 학생 수는, 전체 학생 중에서 집의 거리가 K_2 이하인 학생 수에서 같은 학교를 다니면서 집의 거리가 K_2 이하인 학생 수를 뺀 것과 같다는 사실을 관찰한다. 전체 학생의 집 좌표들을 정렬한 배열과, 각 학교에 대해 해당 학교에 다니는 학생들의 집 좌표들을 따로 모아 정렬한 배열을 준비한 후, 이분 탐색을 사용하면 각 학생의 친구 수를 빠르게 계산할 수 있다.

산책로 - 풀이

작성자: 윤교준

정점과 간선에 양의 가중치가 있는 트리가 주어졌을 때, 서로 다른 두 정점 사이를 잇는 경로의 가중치를 최대화해야 한다. 여기서, 경로 P 의 가중치 $w(P)$ 는 "경로에 속한 간선의 가중치의 총합 $s(P)$ "에서 "경로에 속한 정점의 가중치의 최댓값 $m(P)$ "을 뺀 값으로 정의된다.

부분문제 1

가능한 모든 $\frac{N(N-1)}{2}$ 가지의 경로 각각에 대해, 그 경로의 가중치를 선형에 계산할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 2

경로의 한쪽 끝 정점을 트리의 루트로 고정하자. 한 번의 DFS로, 각 정점에 대해 루트와 그 정점 사이를 잇는 경로의 "간선 가중치 총합"과 "정점 가중치의 최댓값"을 모두 계산할 수 있다. 즉, 한쪽 끝 정점이 고정되어 있을 때, 가능한 모든 $N - 1$ 가지의 경로의 가중치를 선형에 알 수 있다.

이 작업을 모든 정점에 대해 반복하면, 전체 시간 복잡도 $\mathcal{O}(N^2)$ 에 답을 구할 수 있다.

부분문제 3

트리의 지름이 항상 답이 된다. 이는 선형 시간에 알아낼 수 있다.

부분문제 4

주어지는 트리가 선형이다.

일반성을 잃지 않고, 정점의 가중치 A_* 가 모두 서로 다르다고 가정하자. 이는 충분히 작은 양의 실수 ε 을 정한 후, 각 A_i 에 $i\varepsilon$ 만큼 더했다고 생각하면 된다.

고정된 v 에 대해, $m(P_v) = A_v$ 를 만족하는 경로 $P_v = \langle s_v \rightarrow s_v + 1 \rightarrow \dots \rightarrow e_v \rangle$ 의 가중치를 최대화하기 위해서는

- $s_v - 1$ 의 값은 $l < v$ 이고 $A_l > A_v$ 이면서 가장 큰 l 과 같다. (편의상, $A_0 := +\infty$.)
- $e_v + 1$ 의 값은 $r > v$ 이고 $A_r > A_v$ 이면서 가장 작은 r 과 같다. (편의상, $A_{N+1} := +\infty$.)

스택을 사용하여 왼쪽에서 오른쪽으로 배열 A_* 을 순회하면, 각 v 에 대해 그에 대응되는 s_v 를 모두 선형 시간에 계산할 수 있다. e_v 에 대해서도 대칭적으로 계산할 수 있다.

따라서, 최적의 경로의 후보가 되는 N 개의 경로 P_* 를 얻을 수 있다. 각 경로의 $s(P_*)$ 값은 배열 L_* 의 누적합을 통해 상수 시간에 계산할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 5

주어지는 트리가 정점 N 을 중심으로 별 모양을 이룬다. 따라서, 하나의 경로에는 간선이 최대 두 개 존재한다.

하나의 간선으로 구성된 경로들은 자명하다.

일반성을 잃지 않고, $A_1 < A_2 < \dots < A_{N-1}$ 라고 가정하자. 이는 정점 N 을 제외한 나머지 $N - 1$ 개의 정점을 A_* 의 값에 따라 정렬하여 reindexing했다고 생각하면 된다.

두 개의 간선으로 구성된 경로는 항상 $P := \langle x \rightarrow N \rightarrow v \rangle$ ($x < v < N$) 꼴이다. 이때, v 를 고정하면

$$w(P) = L_x + (L_v - \max\{A_v, A_N\})$$

이므로, L_x 가 최대가 되는 $x < v$ 를 선택하는 것이 최적이다.

전체 시간 복잡도는 $\mathcal{O}(N \log N)$ 이다.

부분문제 6

다음 핵심 관찰이 필요하다: 상수 X 를 고정하고, $A_* > X$ 인 정점들을 제거하자. 남아있는 연결 컴포넌트 트리들의 지름을 구해, 그중에서 지름이 최대가 되는 경로를 P_X 라고 하자. $s(P_X) - X$ 를 최대화하는 X 의 값이 X_{opt} 라면, $P_{X_{\text{opt}}}$ 는 문제의 답이 된다.

$m(P_X) \leq X$ 이므로, 문제의 답이 되는 경로 P_{opt} 에 대해, $X = m(P_{\text{opt}})$ 일 때에 $s(P_X) - X$ 도 최대가 된다. 또한, $X \in \{A_1, A_2, \dots, A_N\}$ 만 고려해도 충분하다.

이 부분문제에서는 고려해야 하는 X 의 값이 최대 20가지 이므로, 각 X 에 대해 직접 포레스트를 구성하여 지름을 계산하면 충분하다.

전체 시간 복잡도는 $\mathcal{O}(20 \times N)$ 이다.

부분문제 7

부분문제 6의 핵심 관찰을 적용하자.

X 의 값이 점점 커지면, 포레스트에는 점점 정점과 간선이 추가되면서, 두 트리가 하나의 간선에 의해 하나의 연결된 트리으로 합쳐지는 과정이 반복된다.

두 트리 T_1, T_2 가 간선 e 에 의해 하나의 트리 $T := T_1 \cup \{e\} \cup T_2$ 로 합쳐졌다고 하자. T_1 과 T_2 의 지름이 각각 $P_1 := \langle s_1 \rightarrow \dots \rightarrow e_1 \rangle$, $P_2 := \langle s_2 \rightarrow \dots \rightarrow e_2 \rangle$ 이라고 하자. 트리의 지름의 성질에 의해, T 의 지름은 다음 6개의 경로 중 하나이다:

- 경로 P_1
- 경로 P_2
- 한쪽 끝 정점은 s_1 이거나 e_1 이고, 반대쪽 끝 정점은 s_2 이거나 e_2 인 4개의 경로

따라서, 트리에서 임의의 두 정점 사이의 거리를 효율적으로 계산할 수 있다면, 두 트리가 간선에 의해 하나로 합쳐져서 생기는 새로운 트리의 지름도 효율적으로 계산할 수 있다. 정점 사이의 거리는 최소 공통 조상 알고리즘을 통해 계산하면 충분하다.

즉, A_* 의 값이 작은 정점부터 차례대로 추가하면서, 각 연결 컴포넌트 트리의 지름에 대한 정보를 관리하면, 모든 $X \in \{A_1, A_2, \dots, A_N\}$ 에 대해 $s(P_X)$ 의 값을 효율적으로 계산할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N \log N)$ 이다.

점프 - 풀이

작성자: 박영우

부분문제 1

주어진 N 개의 발판을 정점으로 생각하고 i 번 발판에서 j 번 발판으로 이동할 수 있는 모든 (i, j) 쌍에 대해 간선을 추가하여 방향그래프를 구성할 수 있다. 문제에서 구해야 하는 값은 각 정점에서 도달할 수 있는 정점의 개수이다.

각 정점의 쌍 (i, j) 에 대해 i 에서 j 로 향하는 간선이 있는지 $\mathcal{O}(1)$ 에 판별할 수 있다. 따라서 그래프를 $\mathcal{O}(N^2)$ 에 구성할 수 있고, 각 정점에 대해 해당 정점에서 도달할 수 있는 정점의 집합을 DFS 등의 그래프 탐색 알고리즘을 사용해 $\mathcal{O}(N^2)$ 에 구할 수 있다. 따라서 전체 시간복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 2

i 번 발판에서 도달 가능한 발판들의 집합을 다음과 같이 분류한다.

- L_i : $X_j < X_i$ 이며 i 번 발판에서 도달 가능한 j 번 발판들의 집합
- M_i : $X_i = X_j$ 이며 i 번 발판에서 도달 가능한 j 번 발판들의 집합
- R_i : $X_i < X_j$ 이며 i 번 발판에서 도달 가능한 j 번 발판들의 집합

어떤 i 번 발판에 대해 $R_i \neq \emptyset$ 이라 하자. 이때 k 를 R_i 의 최소 원소라 하자.

임의의 원소 $j \in R_i$ 에 대하여 다음 두 가지 경우 중 정확히 하나만 참이다.

- $X_j > X_k$: i 에서 j 로 가는 경로 중 처음으로 x 좌표가 X_k 를 초과하는 발판을 v , 직전 발판을 u 라 하자. 즉, $X_u \leq X_k < X_v$ 이다.
이때 $X_v - X_u \leq D$ 이므로 $X_v - X_k \leq D$ 이다. 또한 $v \in R_i$ 이므로 k 의 최소성에 의해 $k < v$ 이다.
따라서 k 에서 v 로 직접 이동이 가능하므로 k 에서 v 를 거쳐 j 로 가는 경로가 존재하고, $j \in R_k$ 이다.
반대로 $j \in R_k$ 를 가정하면, R_k 의 모든 원소는 k 를 거쳐 i 에서 도달 가능하므로 $j \in R_i$ 이며, 정의에 의해 $X_k < X_j$ 이다.
따라서 이 경우에서 고려되는 j 들의 집합은 정확히 R_k 이다.
- $X_j \leq X_k$: $X_k \leq X_i + D$ 이므로, $X_i < X_j \leq X_k$ 이면 $X_j - X_i \leq D$ 를 만족한다. 조건에 의해 $i < j$ 이므로, 이를 만족하는 j 는 모두 i 에서 한 번에 이동 가능하다.

따라서 다음 관찰이 성립한다.

R_i 의 각 원소는 다음 중 정확히 하나에 속한다.

- R_k
- $i < j, X_i < X_j \leq X_k$ 인 j 들의 집합

마찬가지로 L_i 에 대해서도 다음과 같은 관찰을 할 수 있다.

$L_i \neq \emptyset$ 일 때, k 를 L_i 의 원소 중 최소 원소라 하자. 그러면 L_i 의 각 원소는 다음 중 정확히 하나에 속한다.

- L_k
- $i < j, X_k \leq X_j < X_i$ 인 j 들의 집합

위 관찰을 통해 각 $i = N, N-1, \dots, 1$ 에 대해 순서대로 R_i, L_i, M_i 를 $\mathcal{O}(N)$ 시간에 구할 수 있다. 따라서 전체 $\mathcal{O}(N^2)$ 시간복잡도에 문제를 해결할 수 있다.

부분문제 3

i 번 발판에서 j 번 발판에 도달할 수 있다면 $i, i+1, \dots, j$ 번 발판 순서대로 이동하는 경로가 존재한다. 따라서 다음과 같이 두 가지의 경우로 나눌 수 있다.

- i 번 발판에서 $i+1$ 번 발판으로 이동할 수 있는 경우: 이 경우 i 번 발판에서 도달할 수 있는 발판은 $i+1$ 번 발판에서 도달할 수 있는 발판들과 i 번 발판 자신이다.
- i 번 발판에서 $i+1$ 번 발판으로 이동할 수 없는 경우: 이 경우 i 번 발판에서는 다른 어떤 발판으로도 이동할 수 없다.

따라서 $N, N-1, \dots, 1$ 번 발판에 대해 순서대로 답을 구해줄 수 있다. 전체 시간복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 4

$X = \max\{X_1, X_2, \dots, X_N\}$ 이라 하자.

S_x 를 $X_i = x$ 인 i 번 발판들의 집합이라 하자.

모든 S_x 를 $\mathcal{O}(N + X)$ 에 구할 수 있다. 각 i 에 대해 R_i 의 최소 원소 k 는 $S_{X_i+1}, \dots, S_{\min\{X, X_i+D\}}$ 의 원소 중 i 초과인 최소 원소이다. 각 집합 S_x 에 대해 i 를 초과하는 가장 작은 원소는 이분 탐색을 통해 $\mathcal{O}(\log N)$ 에 찾을 수 있고, 따라서 k 를 $\mathcal{O}(X \log N)$ 에 찾을 수 있다.

또한 $X_i < X_j \leq X_k$ 이고 $i < j$ 인 j 들의 개수도 같은 방법으로 $\mathcal{O}(X \log N)$ 에 찾을 수 있다. 따라서 각 $i = N, N-1, \dots, 1$ 에 대해 R_i 의 크기를 순서대로 $\mathcal{O}(X \log N)$ 에 구할 수 있다.

같은 방법으로 L_i 의 크기도 같은 시간복잡도에 구할 수 있다.

M_i 의 경우 $i \leq j, X_i = X_j$ 인 발판의 수를 세면 된다. 즉, M_i 의 크기는 S_{X_i} 에서 i 이상인 원소의 개수와 같다. 이는 이분 탐색을 통해 $\mathcal{O}(\log N)$ 에 찾을 수 있다.

따라서 전체 시간복잡도는 $\mathcal{O}(NX \log N)$ 이다.

부분문제 5

S_x 를 부분문제 4와 같이 정의한다. 그렇다면 각 i 에 대해 R_i 의 최소 원소 k 는 S_{X_i+1} 의 원소 중 i 초과인 최소 원소이다. 주어진 인덱스 범위가 크므로 모든 S_x 를 `std::map` 등의 자료 구조로 관리하면 k 를 $\mathcal{O}(\log N)$ 에 구할 수 있다.

또한 어떤 j 번 발판이 $X_i < X_j \leq X_k, i < j$ 를 만족한다면 $X_j = X_k$ 이다. 이러한 j 의 개수도 $\mathcal{O}(\log N)$ 에 셀 수 있다.

따라서 각 $i = N, N-1, \dots, 1$ 에 대해 R_i 의 크기를 순서대로 $\mathcal{O}(\log N)$ 에 구할 수 있다.

비슷한 방법으로 L_i 와 M_i 의 크기도 같은 시간복잡도에 구할 수 있고, 전체 시간복잡도는 $\mathcal{O}(N \log N)$ 이다.

부분문제 6

$i = N, N-1, \dots, 1$ 순서대로 R_i 의 크기를 구할 수 있다.

- i 번 발판에 대해 R_i 의 최소 원소 k 는 현재까지 고려된 발판 ($i+1, \dots, N$ 번 발판)들 중 x 좌표가 X_i 초과, $X_i + D$ 이하인 발판들 중 인덱스가 가장 작은 것이다. 따라서 이를 세그먼트 트리를 통해 관리하면 $\mathcal{O}(\log N)$ 시간에 구할 수 있다. 발판들의 x 좌표 범위가 크므로, 좌표 압축 등의 방법이 필요하다.
- R_i 는 R_k 와 $X_i < X_j \leq X_k, i < j$ 를 만족하는 발판들로 구성되어 있다. 두 번째 종류에 해당하는 발판의 개수는 현재까지 고려된 발판 중 x 좌표가 X_i 초과, X_k 이하인 발판들의 개수와 같다. 이 값 역시 세그먼트 트리를 통해 현재까지 고려된 발판들의 집합을 관리하면 $\mathcal{O}(\log N)$ 시간에 구할 수 있다.

L_i 의 크기는 같은 방법으로 구할 수 있고, M_i 의 크기는 부분문제 5의 방법을 사용하면 같은 시간복잡도에 구할 수 있다.

따라서 전체 시간복잡도는 $\mathcal{O}(N \log N)$ 이다.